Actividad 7: Persistencia en Disco y Estructuras en C++

Sofia Mojica, Gabriel Zacarias, Mateo Roberts y Leonel De Armas

E.E.S.T. N.° 100 "Toy Story" Vicente López

Laboratorio de Programación (LPR), 5.° año

16 de octubre de 2026

Versión del documento: v1.0.0

Actividad 7: Persistencia en Disco y Estructuras en C++

En el desarrollo de software moderno y sistemas embebidos, la gestión eficiente de los datos es un pilar fundamental. Con frecuencia, los programadores junior confunden la memoria volátil (Memoria RAM), cuyo contenido se desvanece al apagar el equipo o finalizar el proceso del programa, con el almacenamiento secundario persistente (discos rígidos o estados sólidos). Esta actividad práctica (Actividad 7) tiene como propósito principal introducir y consolidar el concepto de Persistencia de Datos en C++ mediante el uso de flujos de archivos (`<fstream>`), estructuras heterogéneas (`struct`) y la correcta administración del búfer de entrada.

Como caso de estudio, se implementa un mini juego de "Piedra, Papel o Tijera" contra la inteligencia artificial de la consola, registrando cada partida estructurada en formato CSV dentro del archivo `historial_partidas.txt`. Asimismo, se establece una conexión directa con el Proyecto Integrador Anual (P.I.A.) para la EXPOTÉCNICA 2026, permitiendo extrapolar las técnicas de guardado de registros a logs de hardware, estados de sensores y bases de datos locales.

Integrantes del Equipo y Roles

La Tabla 1 detalla el rol asignado a cada integrante del equipo.

Tabla 1

Integrantes del equipo y roles asignados

Apellido y nombre	Rol
Mojica, Sofia	Lógica del juego: menú, jugada de la IA y reglas de victoria, derrota y empate
Zacarias, Gabriel	Persistencia en disco: `struct Partida`, escritura con `ofstream` (`ios::app`) y lectura con `ifstream`
Roberts, Mateo	Documentación: manuales de usuario y del programador, informe APA v7 y cuestionario técnico
De Armas, Leonel	Control de versiones (GitHub), pruebas de compilación y capturas de evidencia

Desarrollo Técnico y Fundamentos Metodológicos

Modelado con struct

Para representar de manera unificada los atributos de una partida, se diseñó la estructura `Partida`, la cual agrupa variables de diferentes tipos de datos:

- `idPartida` (`int`): Identificador numérico autoincremental único.
- `jugador` (`string`): Nombre o apodo ingresado por el usuario.
- `eleccionJugador` (`string`): Opción seleccionada (Piedra, Papel o Tijera).
- `eleccionIA` (`string`): Jugada generada de forma pseudoaleatoria mediante `rand()`.
- `resultado` (`string`): Resolución lógica de la partida (Victoria, Derrota o Empate).

Persistencia y Flujos de Archivos (<fstream>)

- **Escritura (ofstream con ios::app):** Permite abrir el archivo `src/historial_partidas.txt` en modo de adición (Append). Esto garantiza que las nuevas partidas se escriban al final del archivo sin sobrescribir ni eliminar los registros previos acumulados en ejecuciones anteriores.
- **Lectura (ifstream):** Abre el archivo en modo de entrada para recuperar secuencialmente cada línea, separando los campos mediante el delimitador de comas (formato CSV) a través de la función `getline`.

Cuestionario Técnico de Control

Pregunta 1: RAM vs. Disco

¿Por qué si cerrás la ventana de consola y volvés a abrir el ejecutable, la Opción 2 sigue mostrando el historial de partidas anteriores?

Respuesta: Porque los datos de las partidas son volcados de forma inmediata al almacenamiento secundario (disco rígido/sólido) a través del flujo `ofstream`. El disco es un medio no volátil, lo que significa que la información persiste independientemente de que el proceso del programa finalice o se cierre la sesión de la consola.

Pregunta 2: Uso de ios::app

¿Qué sucedería con el archivo `historial_partidas.txt` si en la función `guardarEnDisco` utilizaras `ofstream archivo("historial_partidas.txt");` sin incluir la opción `ios::app`?

Respuesta: Por defecto, el constructor de `ofstream` sin modificadores abre el archivo en modo de truncamiento (`ios::out`), lo que borraría por completo todo el contenido previo del archivo cada vez que se guarde una nueva partida, conservando únicamente la última jugada realizada.

Pregunta 3: Manejo del Búfer (cin.ignore())

¿Por qué es necesario invocar `cin.ignore()` antes de leer el nombre del jugador con `getline(cin, p.jugador)`?

Respuesta: Cuando el usuario ingresa una opción numérica mediante `cin >> opcion`, presiona la tecla Enter, dejando un carácter de salto de línea (`\n`) pendiente en el búfer de entrada. Si no se utiliza `cin.ignore()`, el `getline` posterior detecta inmediatamente ese `\n` remanente, interpretándolo como una cadena vacía y salteando la lectura del nombre del jugador.

Pregunta 4: Agrupación con struct

¿Qué ventajas ofrece la estructura `Partida` para organizar las variables del juego frente a pasar 5 variables sueltas a cada función?

Respuesta: La estructura promueve la modularidad y el principio de encapsulamiento, permitiendo agrupar lógicamente datos relacionados bajo una sola entidad. Esto simplifica la firma de las funciones, reduce el acoplamiento y facilita la escalabilidad del código al momento de transmitir o almacenar múltiples atributos de forma conjunta.

Pregunta 5: Aplicación al P.I.A.

Si adaptaran este módulo de almacenamiento a su Proyecto Integrador Anual, ¿qué datos o eventos de su prototipo guardarían en el archivo de texto?

Respuesta: En el contexto de la EXPOTÉCNICA 2026, esta arquitectura de persistencia se puede aplicar para almacenar registros de sensores en tiempo real

(temperatura, humedad, presencia), bitácoras de auditoría de usuarios (logs de acceso) o estados críticos de control de stock y fichaje de turnos en formato CSV o plano.

Conclusión

La implementación de estructuras de datos combinada con flujos de archivos en C++ otorga a los estudiantes las herramientas fundamentales para dotar a sus aplicaciones de robustez y persistencia. Este proyecto cumple con los objetivos pedagógicos de la materia Laboratorio de Programación y sienta las bases técnicas indispensables para afrontar con éxito el desarrollo del Proyecto Integrador Anual en la EXPOTÉCNICA 2026.

Enlace al Repositorio

El código fuente y la documentación de la actividad están disponibles en el siguiente repositorio de GitHub:

`https://github.com/matuteXEN6/ACTIVIDAD-7-MINI-JUEGO-PIEDRA-PAPEL-O-TIJERA-CON-PERSISTENCIA-DE-DATOS.git`